

Final Project: Collaborative Dual-Elevator System over SPI IPC

Course: Embedded Systems Level 1 (Cortex-M4)

Project Type: Distributed Reactive System & Hardware IPC Challenge

1. Project Objective

Design and implement a coordinated, two-elevator control system serving 4 floors using **two independent STM32 (Cortex-M4) microcontrollers**. You must develop a robust **Full-Duplex SPI Inter-Processor Communication (IPC)** link to synchronize states and implement a **Master-driven Task Allocation Algorithm** to optimize building traffic.

2. Hardware Architecture & IO Mapping

The system consists of a **Master MCU** (Dispatcher + Elevator A) and a **Slave MCU** (Elevator B).

A. Inputs (Asynchronous Events - EXTI only)

Input Device	Master MCU (Board A)	Slave MCU (Board B)
Cabin Floor Buttons	4 Buttons (Internal Requests)	4 Buttons (Internal Requests)
Emergency Stop	1 Button (Highest NVIC Priority)	1 Button (Highest NVIC Priority)
Hallway Floor Calls	6 Buttons (U1, D2, U2, D3, U3, D4)	None (Received via SPI)
Floor Sensors	4 Sensors (Position Tracking)	4 Sensors (Position Tracking)

B. Outputs (Actuation & Monitoring)

- **Motor Simulation:** 1 x LED per board using **PWM Mode**. Duty cycle represents speed (0% Stop, 20% Slow, 100% Full).
- **Telemetry:** UART connection to PC (500ms non-blocking status updates).

3. The IPC Link (SPI Protocol Requirements)

The two MCUs must be physically linked via a 4-wire SPI bus (SCK, MOSI, MISO, CS).

- **Mode:** Full-Duplex.
- **The Slave Challenge:** You must implement a non-blocking driver where the Slave **pre-loads** its status into the TX register *before* the Master initiates a transfer.
- **Packet Structure:** You must define a fixed-length frame (e.g., 8 bytes) including a **Header (0xA5)**, **State Data**, and a **Checksum**.
- **Synchronization:** State exchange must occur at a high enough frequency (e.g., every 50ms) to ensure the Master's decisions are based on real-time data.

4. Mandated Task Allocation Algorithm

The Master MCU is the "Brain." When a **Hallway Floor Call** is pressed, the Master must run the following logic to decide which elevator to dispatch. **Simple Round-Robin or "Nearest Idle Only" is forbidden.**

Scenario	Decision Rule
Comm Fault	If SPI times out, Master takes all calls; Slave enters "Independent/Emergency" mode.
Immediate	If an elevator is already at the floor and is IDLE , assign to it.
Perfect Match	Elevator is moving <i>towards</i> the floor in the <i>same</i> direction. (e.g., At Floor 1, going UP; Call is Floor 2, UP). Assign here.
Passed Match	Elevator is in the same direction but has already passed the floor. Lower Priority.

Opposite Dir	Elevator is moving away from the call direction. Do NOT assign until it finishes its current path.
Idle	If no directional match exists, assign to the nearest IDLE elevator.

5. Software Engineering Requirements

Success is dependent on applying the advanced concepts from **Lecture 1-7** and **Slide 10 (Self-Study)**:

1. **Concurrency:** You must use the **volatile** keyword for all flags shared between ISRs and the main FSM.
2. **Critical Sections:** Protect SPI RX/TX buffers using an **Enter_Critical()** / **Exit_Critical()** mechanism to prevent data corruption during read-modify-write operations.
3. **FSM Design:** Both elevators must be implemented as Finite State Machines (**IDLE**, **MOVING_UP**, **DOORS_OPEN**, etc.).
4. **Non-Blocking Logic:** Polling is only allowed for the UART debug output. All other timing must be handled by Timers and Interrupts.

6. Laboratory Preparation (Pre-requisite)

Before coding, each group must submit:

- **Register Map:** A table of all used peripherals (RCC, GPIO, EXTI, NVIC, TIM, SPI, UART) and their base addresses.
- **PWM Math:** Calculations for **PSC** and **ARR** to achieve a specific frequency (e.g., 10kHz) for smooth LED dimming.
- **Packet Definition:** A diagram of your 8-byte SPI frame.

7. Evaluation Rubric

Category	Weight	Criteria
IPC Reliability	30%	Robust SPI data exchange, no "glitched" data, proper checksum usage, non-blocking Master/Slave drivers.

Algorithm Quality	20%	Adherence to the mandated directional optimization rules. System efficiency.
Race Condition Handling	20%	Correct use of volatile , atomic access, and critical sections. No "hanging" or "crashing" during high-frequency button presses.
Architecture	15%	Clean separation of Drivers (HAL) and Application (FSM). Use of a hardware-based scheduler.
Functionality	15%	Emergency stop works instantly; PWM speed ramps; UART telemetry is accurate.

Bonus (+5 points): Implement **DMA (Direct Memory Access)** for the 500ms UART telemetry report to ensure zero CPU overhead for monitoring.